

Oficina Mecânica API

Autenticação na borda, rede privada, ambientes segregados e observabilidade.

Danilo Canato — RM 372411 — Discord: @canatodanilo

Guilherme Fumagali Marques — RM 372421 — Discord: @gfumafali

Situação ao final da Fase 2

Três dos cinco requisitos de infraestrutura desta fase já estavam atendidos.

JÁ EXISTIA

Cluster e banco

EKS com HPA por CPU e memória.

RDS PostgreSQL 16 gerenciado, sem acesso público.

Tudo provisionado por Terraform, com state em S3.

JÁ EXISTIA

Esteira

Build, testes, SonarCloud e deploy automático no cluster.

OIDC para a AWS, com uma única role de administrador.

PENDENTE

Borda e identidade

Sem API Gateway: a proteção era feita apenas pelo Spring Security.

A aplicação emitia o próprio token, com um usuário administrador em memória.

A implementação concentrou-se em autenticação, segregação de repositórios e observabilidade.

Requisitos da Fase 3

Os requisitos tratam de capacidade operacional e não acrescentam funcionalidades de negócio.

- 01** Autenticação por CPF em função serverless, atrás de um API Gateway
- 02** Quatro repositórios separados, cada um com CI/CD e deploy automático
- 03** Infraestrutura de nuvem inteira declarada em Terraform
- 04** Observabilidade com plataforma de mercado, e dashboards de negócio
- 05** Documentação arquitetural formal: RFCs, ADRs, C4, sequência e DER

Foco em segurança, operação e visibilidade do sistema.

Emissão de tokens na função serverless

Principal alteração na aplicação nesta fase.

Cliente → API Gateway → Lambda: valida CPF · consulta status no banco · assina JWT

Cliente → API Gateway → Lambda authorizer valida na borda → cluster

REMOVIDO

Da aplicação

AuthController, UserDetailsConfig, RefreshTokenService e três DTOs.

O JwtService perdeu a geração e ficou só com verificação e leitura de claims.

MOTIVAÇÃO

Um único emissor

Um lugar para rotacionar a chave.

Uma semântica de claims.

Um ponto de auditoria.

CONSEQUÊNCIAS

Sessão de 15 minutos

Sem refresh token: o cliente se autentica novamente com o CPF quando o token expira.

O sub do token é o UUID do cliente, e não o CPF.

API Gateway

Requisições sem token válido são rejeitadas antes de chegar ao cluster.

PROTEGIDO

ANY /api/**

Lambda authorizer valida a assinatura, com cache de 300 s.

Funcionário acessa a gestão; cliente, só as próprias OS e veículos.

SEM CPF

Rotas de terceiros

Consulta de status por ID opaco.

Aprovação externa de orçamento, chamada por sistema de terceiro com token de uso único.

ESCOLHA

Lambda authorizer

O JWT authorizer nativo exige OIDC e JWKS.

Ele não valida HMAC, pois a assinatura simétrica não possui chave pública.

Decisão registrada no diagrama de sequência e na especificação do gateway.

Repositórios e ambientes

Quatro repositórios com pipelines próprios, integrados pelo SSM Parameter Store.

tech-challenge-1

oficina-api · Spring Boot ·
histórico das 3 fases

oficina-auth-lambda

SAM Java 21 · duas funções

oficina-infra-k8s

Rede, EKS, ECR, gateway, New
Relic

oficina-infra-db

RDS PostgreSQL gerenciado

INTEGRAÇÃO: SSM PARAMETER STORE

Cada repositório publica seus outputs sob /oficina/<ambiente>/ e consome os dos demais por data source, sem terraform_remote_state, de modo que nenhum precise de permissão sobre o state de outro.

develop → homologação · main → produção com aprovação · cada ambiente com VPC, cluster, banco, state e segredos próprios
· uma role por repositório, com menor privilégio

Rede privada e acesso à internet

Nós e banco migrados para subnets privadas. Opções avaliadas para a saída de internet:

NAT Gateway	US\$ 0,045/h	~US\$ 33/mês
VPC interface endpoints	US\$ 0,01/h × 4 endpoints × 2 AZs	~US\$ 58/mês
NAT instance t4g.nano + EIP	US\$ 0,0042/h + US\$ 0,005/h	~US\$ 7/mês

Interface endpoints são cobrados por endpoint e por zona de disponibilidade e não atendem ao endpoint público do New Relic.
Em subnet privada, os nós deixam de usar IPv4 público, cobrado desde fev/2024 (US\$ 7,30/mês).

Banco de dados

PostgreSQL 16 em RDS mantido, com alternativas avaliadas na RFC-002.

AVALIADO

DynamoDB

Escala melhor com Lambda, sem limite de conexões.

Exigiria desnormalizar e reescrever a camada de persistência.

AVALIADO

Aurora Serverless v2

Compatível com PostgreSQL, com escala automática.

A capacidade mínima custa mais que a instância utilizada.

ESCOLHIDO

PostgreSQL 16 · RDS

Domínio relacional: integridade referencial em 5 tabelas e junções nas consultas de OS.

Schema já depende de pgcrypto e gen_random_uuid().

Ajustes no modelo: V6 → clientes.status, consultado pela Lambda, e 2 índices · V7 → data_ultima_transicao, para medir a permanência por status

Observabilidade

New Relic: plano gratuito permanente, enquanto o Datadog oferece avaliação de 14 dias.

Latência das APIs	Agente Java · p95 e p99 por endpoint
CPU e memória do K8s	Integração Kubernetes por DaemonSet
Healthchecks e uptime	Monitor sintético · 3 localizações · a cada 5 min
Falhas no processamento de OS	Condição NRQL sobre métrica customizada
Logs JSON com correlação	Spring Boot 3.4 em formato ECS · envio pelo agente com trace.id

Logs enviados pelo agente Java com o trace.id da transação; o Fluent Bit fica desligado para evitar linhas duplicadas.

Dashboards de negócio

Métricas customizadas emitidas pela aplicação, complementares aos dados do APM.

MÉTRICA CUSTOMIZADA

Volume diário de OS

oficina.os.abertas

Emitida no caso de uso, representando o evento de negócio.

MÉTRICA CUSTOMIZADA

Tempo médio por status

oficina.os.duracao_status

O agregado mede a permanência no estado anterior, e o adaptador de persistência publica o evento.

MÉTRICA CUSTOMIZADA

Erros de integração

oficina.integracao.falhas

Segmentada por integração e motivo: e-mail e aprovação externa.

6 alertas criados por Terraform, no padrão AWS-OficinaAPI-<Amb>-<Recurso>-<Sintoma>-<Severidade>, cada um com link para o runbook correspondente.

Documentação arquitetural

3 RFCs · 15 ADRs · 3 diagramas C4 · 2 de sequência · 1 DER · 6 runbooks

ANTES DA DECISÃO

RFC

Registra a proposta, os comentários dos revisores e sua resolução.

Temas: nuvem, banco e autenticação.

DEPOIS DA DECISÃO

ADR

Registra a decisão, as alternativas e as consequências.

Após aceita, não é editada; mudanças geram nova ADR.

OBSERVAÇÃO

RFC no material

O material da fase aborda ADR, DAS, HLD/LLD e C4, mas não RFC.

A convenção adotada está documentada.

A ADR-011, retroativa, expôs divergência entre HPA e node group e previu o risco da métrica de memória; o teste de carga confirmou o risco e a ADR-015 registrou a correção.

Resultados

O que foi entregue nesta fase.

ENTREGUE

Operação

Autenticação por CPF fora da aplicação, com papéis de cliente e funcionário.

Gateway na borda, rede privada e dois ambientes segregados.

ENTREGUE

Visibilidade

Métricas de negócio.

Logs estruturados correlacionados aos traces.

Alertas associados a runbooks.

ENTREGUE

Qualidade

147 casos de teste na aplicação e 44 na Lambda, com limite mínimo de cobertura.

Contrato OpenAPI verificado, tflint e checkov na infraestrutura.

Ambiente validado de ponta a ponta: provisionamento, carga e destruição.

DEMONSTRAÇÃO

Fluxo principal em homologação

- 1 Merge de um PR na develop: a esteira começa a rodar.
- 2 Funcionário autentica por CPF e abre uma ordem de serviço.
- 3 Cliente autentica por CPF e recebe o JWT.
- 4 Cliente consulta as próprias ordens com o token.
- 5 Esteira: testes, imagem no ECR e deploy automático no EKS.
- 6 Carga com k6: dashboard de negócio e HPA criando réplicas.
- 7 Busca pela ordem criada: log estruturado e trace distribuído.

Obrigado.

github.com/Guilherme-Fumagali/tech-challenge-1

github.com/Guilherme-Fumagali/oficina-auth-lambda

github.com/Guilherme-Fumagali/oficina-infra-k8s

github.com/Guilherme-Fumagali/oficina-infra-db